Hands-On with Rumoca

James Goppert

Purdue University · School of Aeronautics and Astronautics

Georgia Tech · 10:00–13:00

github.com/CogniPilot/rumoca_workshop_amc2026

Run the core. Build something extra.

Core: edit a model, fit data, optimize a trajectory

Apply Rumoca to a problem you care about

Optional idea: a Three.js Artemis-style launch

Audience favorite wins a CogniPilot stocking cap

Enter by 12:40 · up to 5 demos · CHALLENGE.md · APP_PROMPT.md

Make declared physics reusable

A physical model should travel with the question.

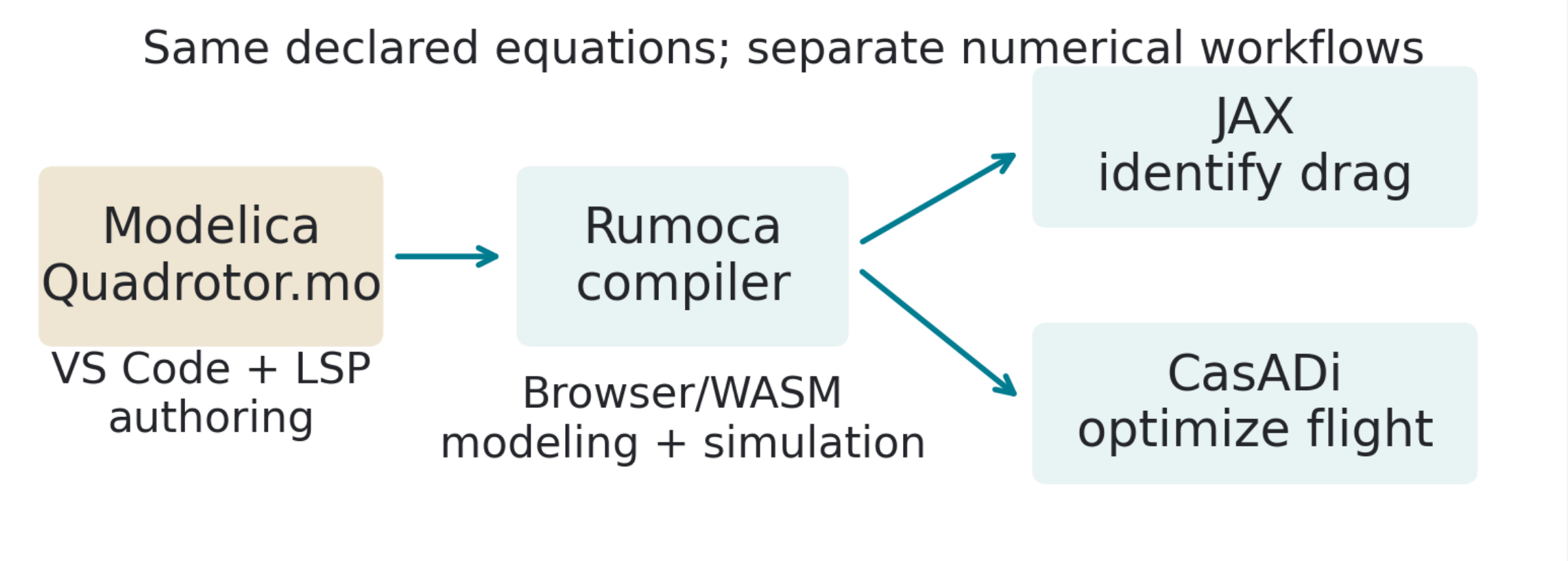
Simulation → identification → control → experiment

Open tools and explicit mathematics help us compare results.

Rumoca connects Modelica to modern computational tools.

Declare once. Ask different questions.

Same Quadrotor.mo equations, two generated Python representations



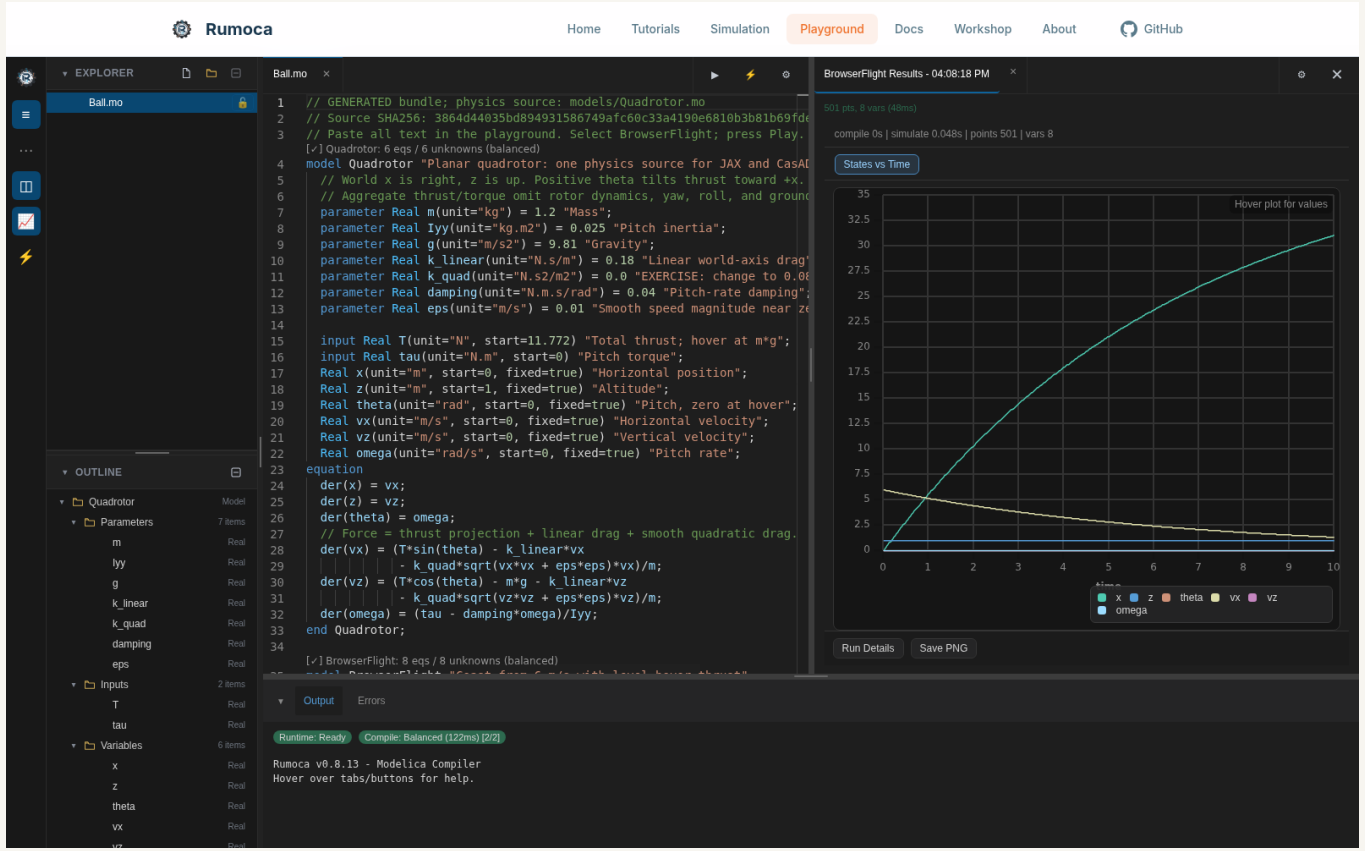
Modelica declares physics · Rumoca compiles · backends compute.

Everyone gets a browser plot

Open the playground

Paste the complete browser bundle

Gear → BrowserFlight → Play



You should see BrowserFlight Results and a plot.

Choose a full-workshop route

Browser: modeling and predictions; no account

Codespaces: Python + VS Code; login and allowance

Local uv: Python + VS Code; pinned CPU packages

Pair or screenshare: keep working if setup fails

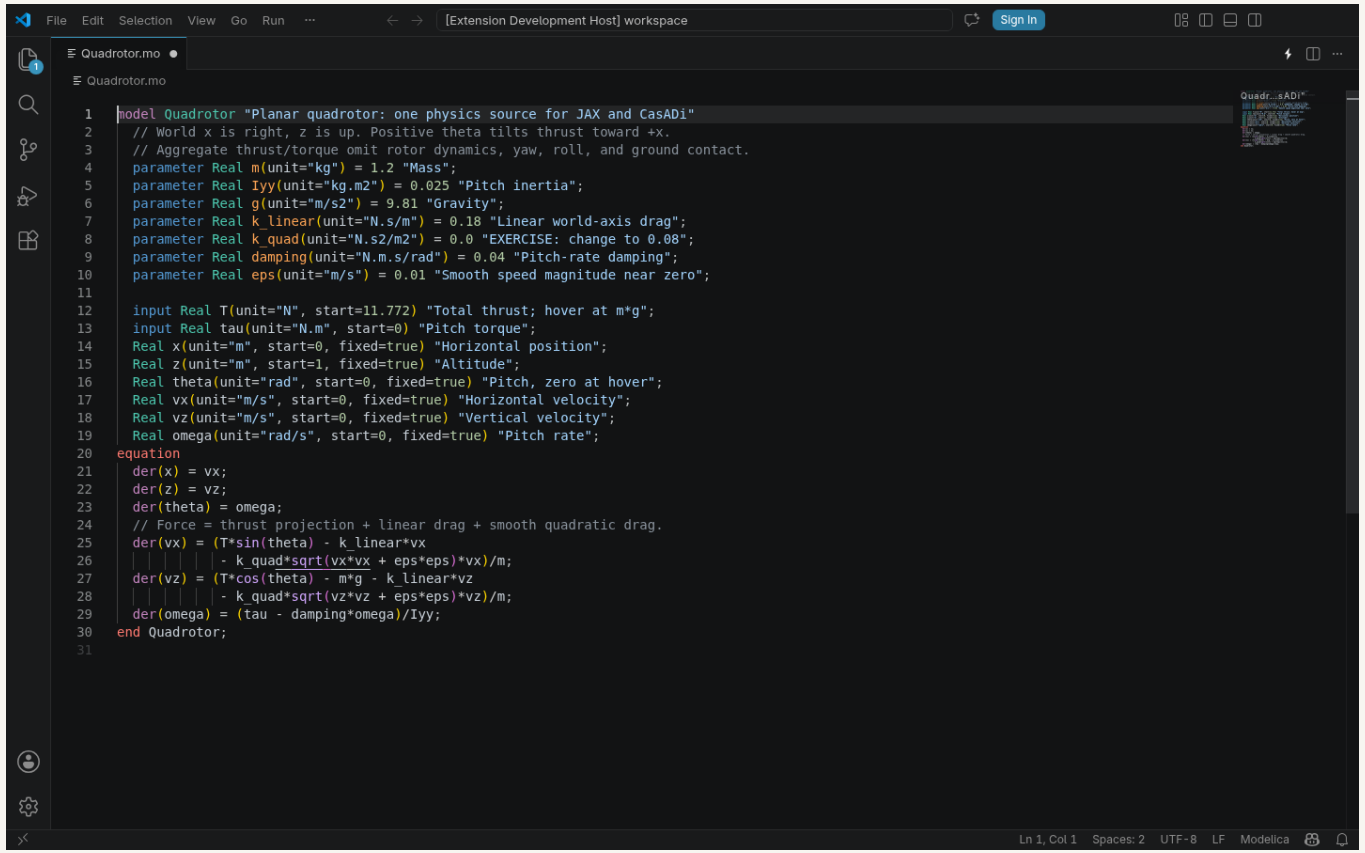
Switch route after ten minutes of setup trouble.

Edit with feedback. Check the pipeline.

Open models/Quadrotor.mo

Hover m; try sin completion

Run the smoke test; open results/live




```
uv run --frozen python scripts/smoke_test.py
```

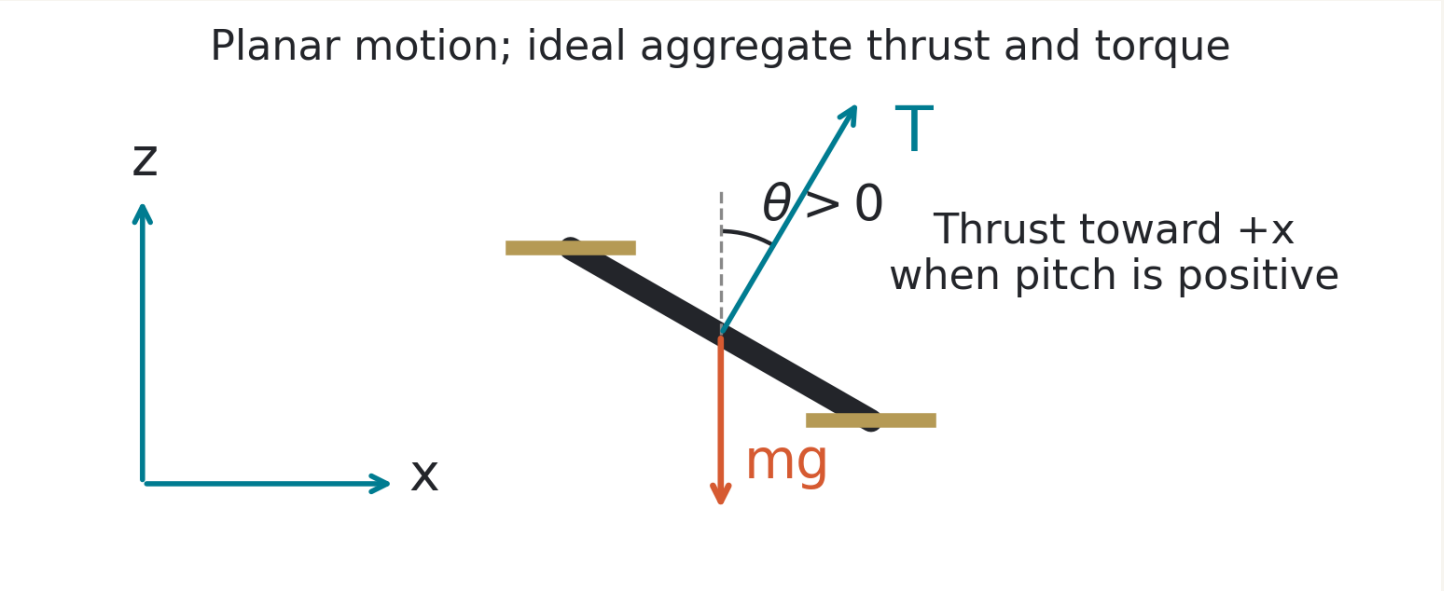
WORKSHOP PASS · model editor ready

A planar quadrotor we can inspect

World x right; z up

Positive pitch tilts thrust toward +x

Hover: T = mg = 11.772 N



Find velocities, pitch rate, thrust and torque in the model.

Your first engineering change: drag

Change k_quad = 0.0 to 0.08, save, then simulate.

$$D(v) = -k_{\text{linear}} v - k_{\text{quad}} \sqrt{v^2 + \varepsilon^2}\, v$$

Linear: N s/m Quadratic: N s²/m² ε = 0.01 m/s

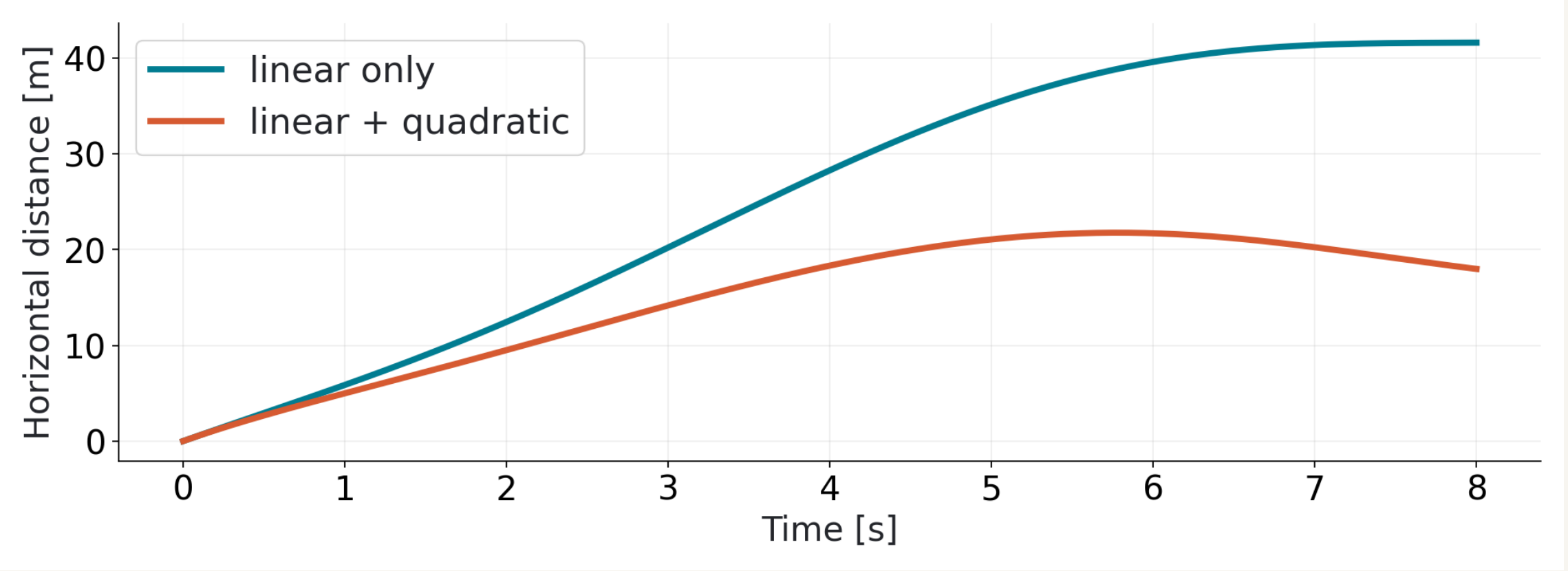
Nonnegative coefficients ⇒ drag opposes motion

```
uv run --frozen python scripts/simulate.py
```

What changes at high speed? What about negative velocity?

More drag, less distance

Reference at 8 s: 41.60 m → 17.97 m



Compare your current default with the two reference cases.

Break · back at 11:10

Keep your editor and terminal open.

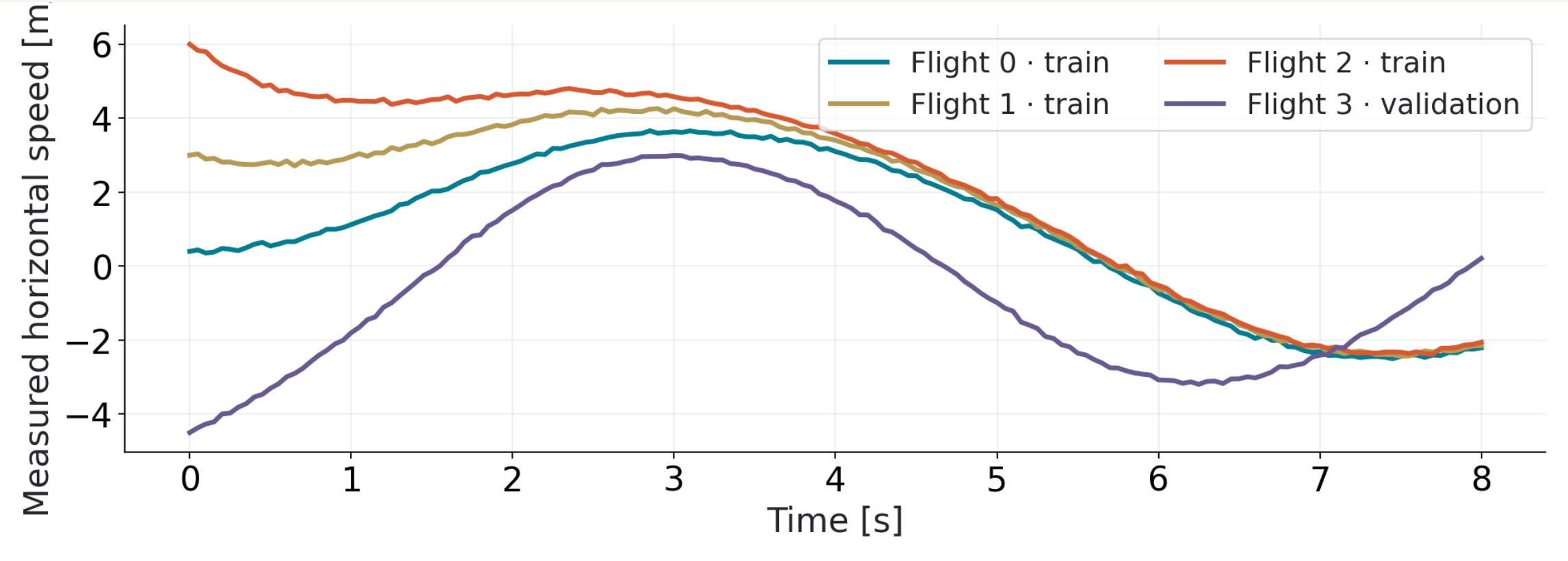
Recovery: results/reference/simulation.png

Next: fit drag from noisy flight data.

Worksheet and commands: WORKSHOP.md · scan the opening QR.

Known experiment. Unknown coefficients.

Three training flights; one held-out flight. 20 Hz, 8 s each.



Speed variation helps separate linear and quadratic drag.

Differentiate the prediction error

Positive parameters; noise-scaled loss; bounded optimization



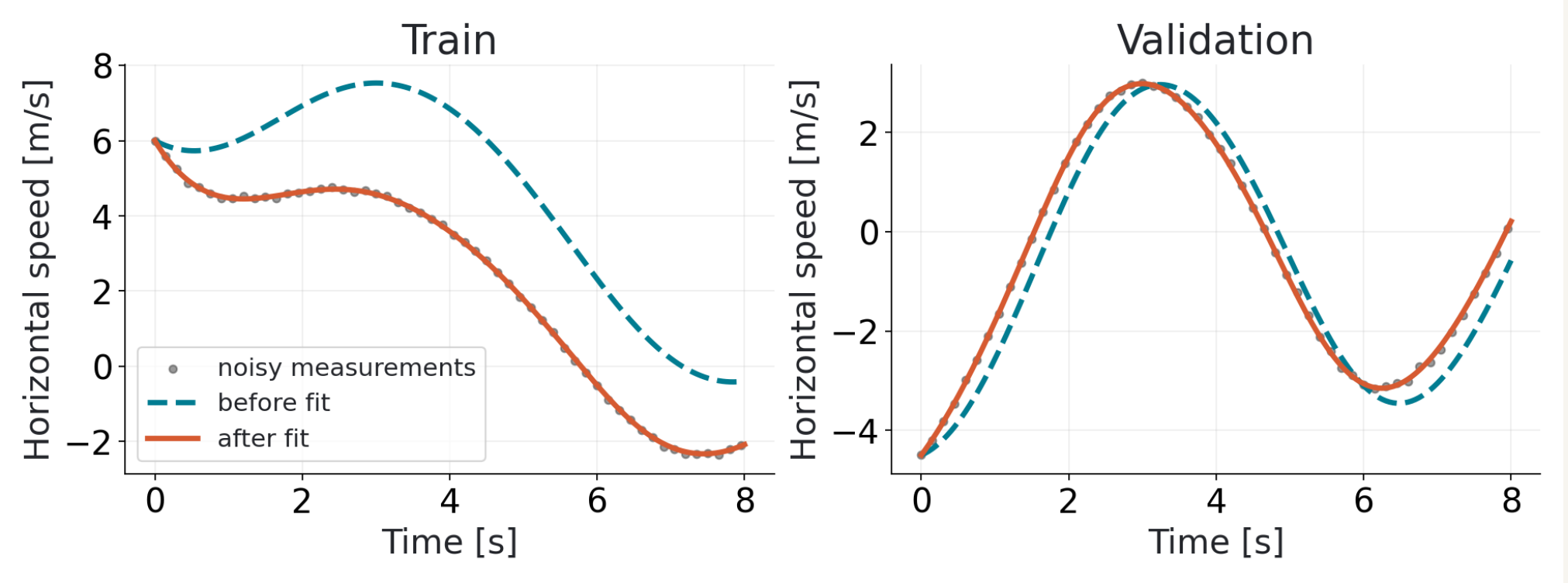
$$L = \mathrm{mean}\left[\left(\frac{\hat{y} - y}{\sigma}\right)^2\right]$$

$$k_l/k_{l,\mathrm{ref}} = e^a, \quad k_q/k_{q,\mathrm{ref}} = e^b$$

JAX provides gradients. SciPy updates the coefficients.

Run the fit. Test the held-out flight.

Open fit_drag.png and fit_drag.json; compare before and after.




```
uv run --frozen python scripts/fit_drag_jax.py
```

Two coefficients · lower loss · improved held-out prediction

Check your result. Keep building.

Open your before-and-after prediction plot

Compare the held-out flight and fitted values

Stretch: rerun from another starting guess

App builders: keep your prototype moving

Fit complete? Continue your app or explore the optional checks.

Break · back at 12:10

Keep results/live/fit_drag.json.

Next: use your coefficients to optimize a flight.

Recovery: results/reference/fit_drag.png

Same Modelica source; now compile to CasADi.

Move between hovers, respect limits

(0, 1) m → (4, 2) m in 6 s

Track target; limit effort and control changes

Bound thrust, torque, pitch and altitude



T: 0–20 N τ: ±0.35 N m

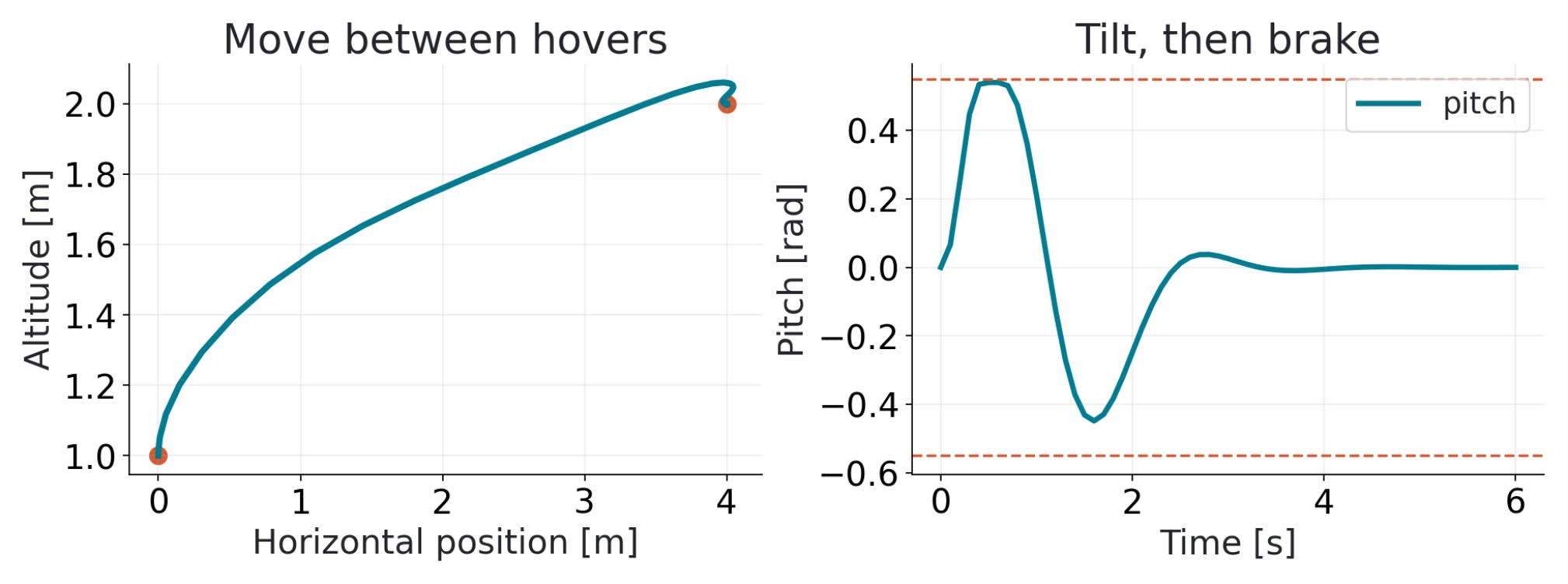
z: 0.25–4 m |θ| ≤ 0.55 rad

60 shooting intervals · node pitch margin · independent reintegration

Identified drag is an input to this optimization.

Solve. Inspect. Explain the braking.

Open optimization.png and optimization.json.

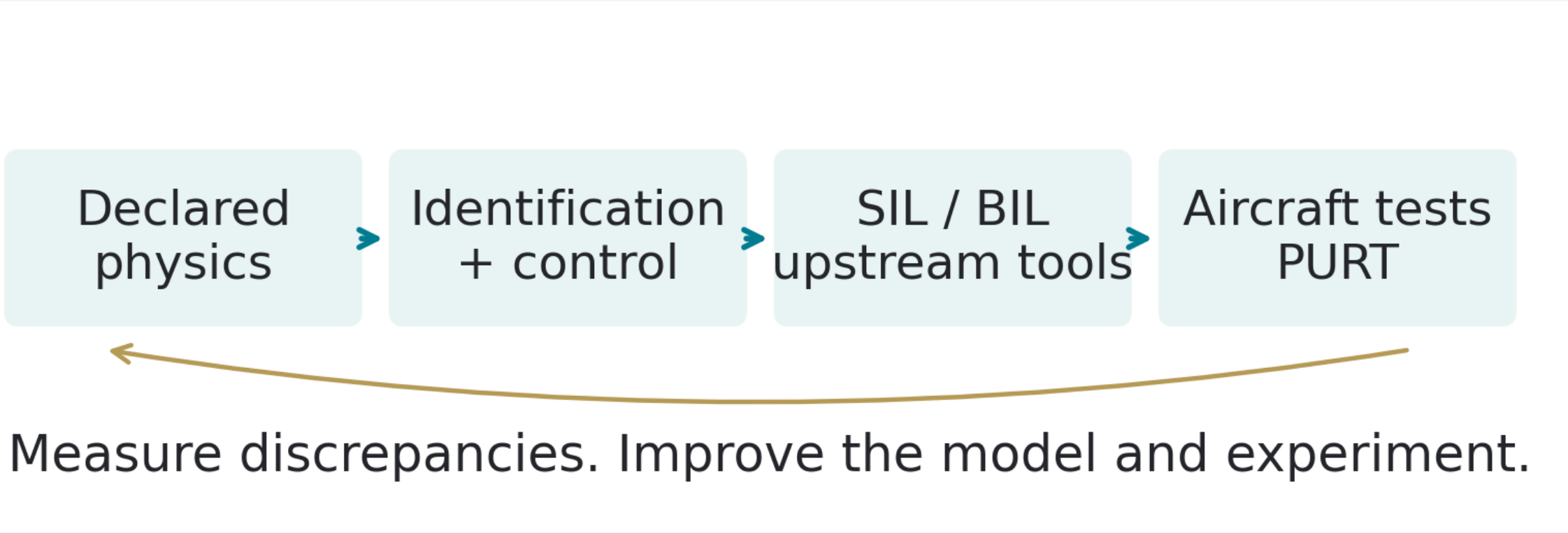



```
uv run --frozen python scripts/optimize_casadi.py
```

Solve_Succeeded · limits respected · arrive near rest

Bring the model back to the experiment

PURT: aircraft experiments, racing and security applications



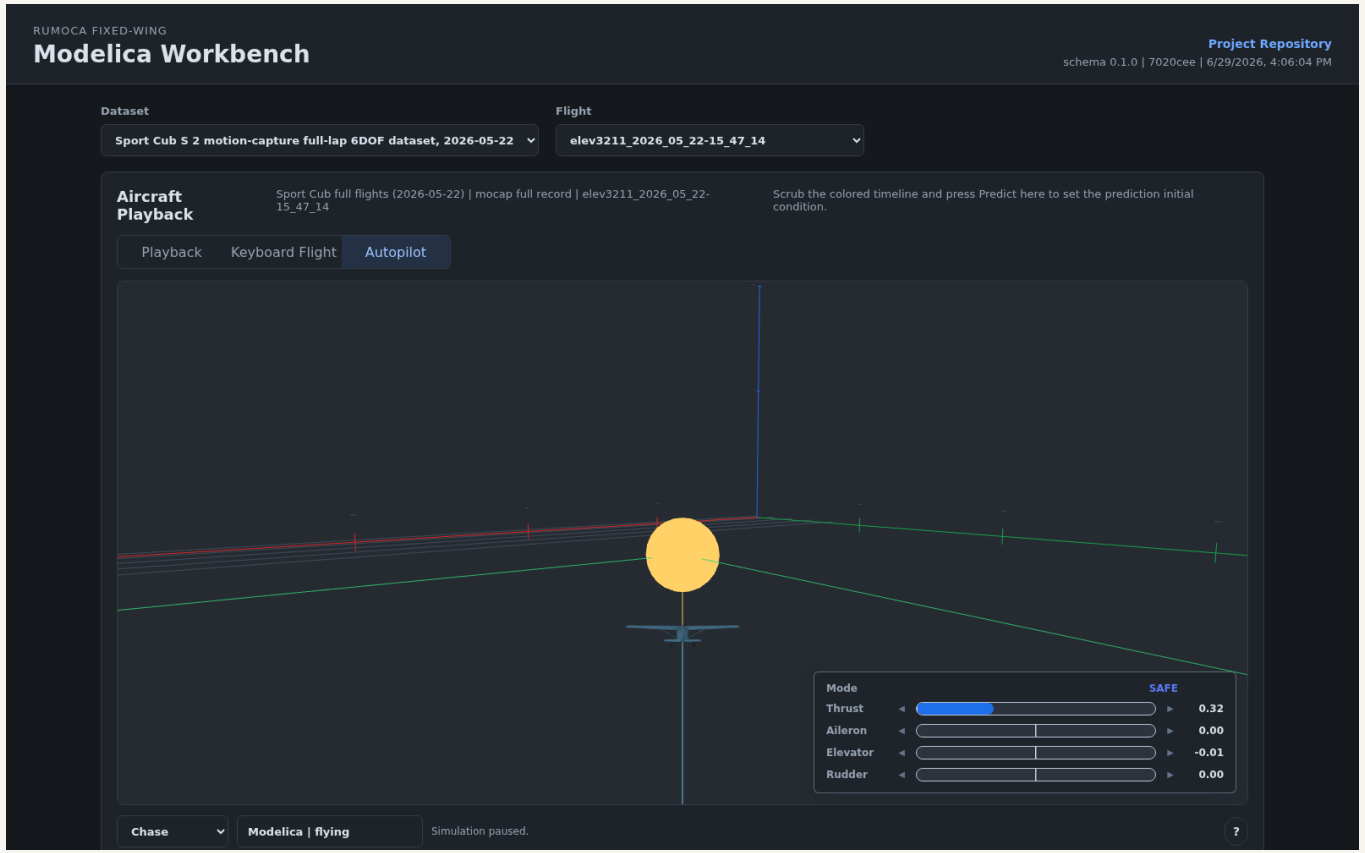
Experimental validation is the next engineering step.

Explore the fixed-wing workbench

Modelica flight simulation + autopilot

Try 50 Hz → 20 Hz, restart, then observe

Also: slam_web (in development), mocap_sysid



Live links + short experiments: BROWSER_APPS.md

Show your app. Vote for the cap.

Up to 5 demos · 45 seconds each · your GitHub

One checked result · agent used · elapsed build time

Audience favorite wins a free CogniPilot stocking cap

Random draw if more than 5 ready entries → audience hand vote

Questions + your next problem

Bring a modeling, identification or optimization problem

Share a reproducible model and a measurable result

Collaborate with Purdue and the open-source community

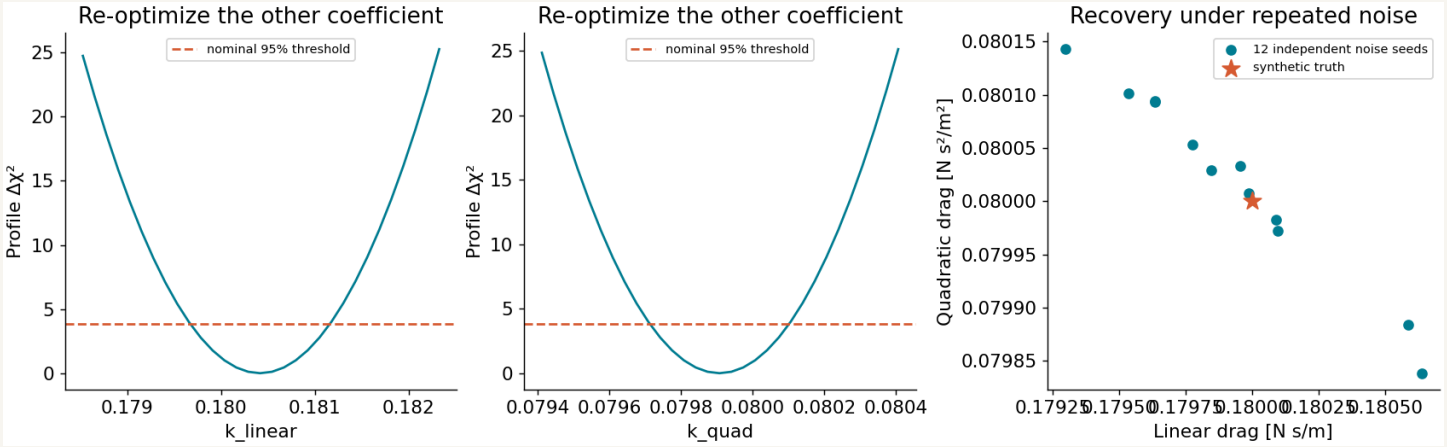
github.com/CogniPilot/rumoca_workshop_amc2026

Exact-real proofs and numerical evidence

Lean: equation equality and invariants

Audit: gradients, profiles, integration

Papers: model and method choices



None of these certify arbitrary edits or real flight.

Primary papers behind the approach

Faessler et al., RA-L 2018: rotor-drag physics and excitation

Andersson et al., 2019: CasADi AD and multiple shooting

Raue 2009; Wieland 2021: practical identifiability and profiles

World-axis teaching drag differs from body-axis rotor aerodynamics.

Research questions worth exploring

Vision + SLAM + estimation + physics in one declared system

Richer browser / FMI / WebAssembly workflows

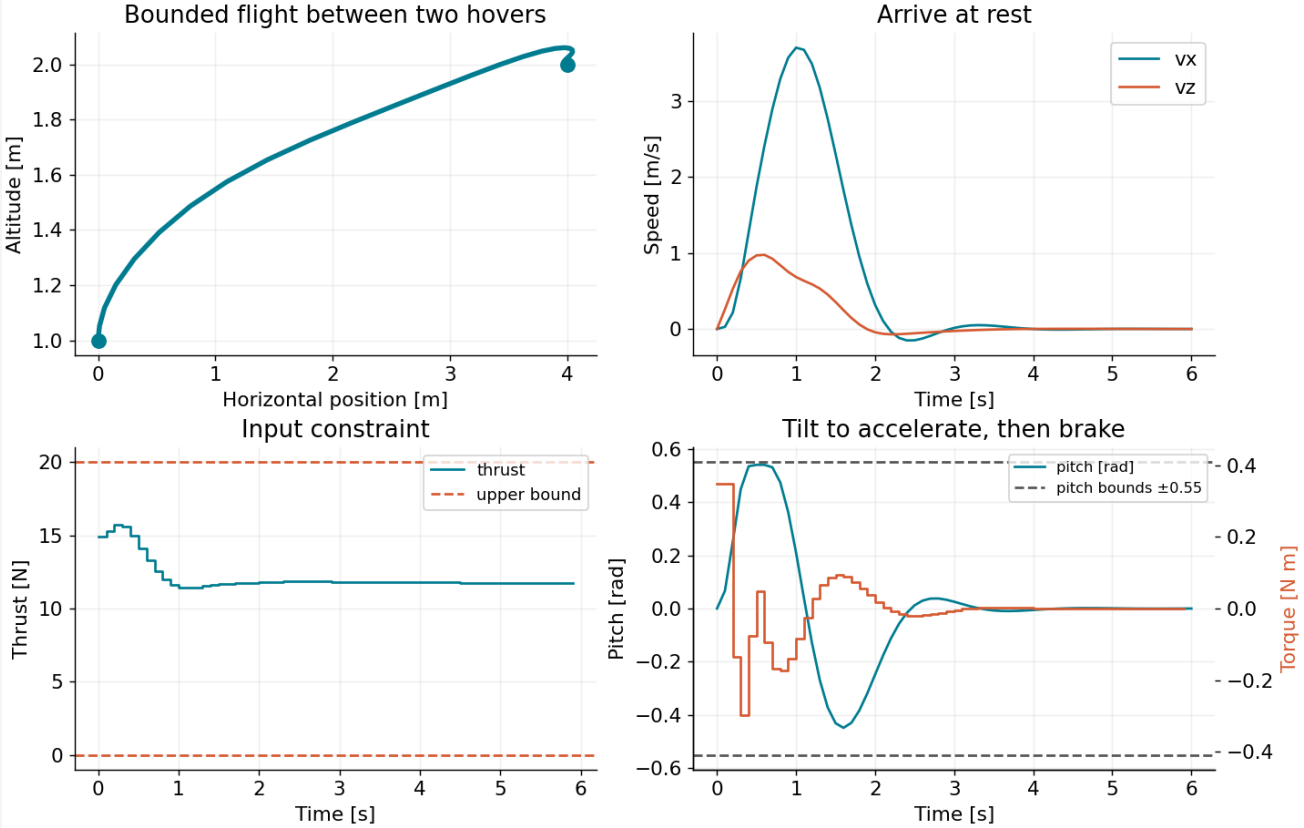
Compiler differentiation and larger symbolic systems

Partition work across CPU, GPU and embedded hardware

Bring a falsifiable question and a measurable benchmark.

Numerical constraints need validation

Reintegrate the inputs independently; inspect errors by physical unit.



Small defects, pitch margin and fine rollout are separate checks.